Purpose-Based Protein Models: Compiled Probes for the Categorical Protein Database

Domain-Specific Neural Compilation from Natural Language to Geometric Operations on S-Entropy Space

Kundai Farai Sachikonye
AIMe Registry for Artificial Intelligence
Experimental Bioinformatics
Maximus-von-Imhof-Forum 3
85354 Freising, Germany
kundai.sachikonye@bitspark.com

April 1, 2026

Abstract

We establish a mathematical framework for *compiled probes*: domain-specific neural models that translate natural language protein queries into executable sequences of geometric operations on S-entropy space. These models store no protein data. Instead, they encode the *compilation mapping* from high-level questions ("Will this sequence fold?", "Is this mutation pathogenic?", "What binds this active site?") to the geometric primitives—Identify, Similar, Predict, React—that the categorical protein database resolves.

The categorical protein database is an *empty dictionary*: it contains no stored structures, no precomputed answers, and no empirical data. It synthesizes protein properties on demand from the geometry of bounded phase space through ternary addressing and trajectory completion. The models presented here are the *compilers* for this engine. Each model has learned, through knowledge distillation from a teacher with access to the full theoretical corpus, how to formulate the right query against the empty dictionary.

We define four probe architectures—Folding, Binding, Disease, and Design—each targeting a distinct class of protein queries. We prove that the compilation problem for protein queries is PAC-learnable (Theorem 2.11), that any valid protein query decomposes into a bounded sequence from five primitive operations (Theorem 3.6), and that LoRA adaptation of rank $r \geq K + L$ suffices for the compilation mapping (Theorem 4.3). We derive the composite training objective that enforces generation fidelity, type safety, S-entropy conservation, and trajectory convergence, and prove its completeness (Theorem 6.2).

The end-to-end pipeline is: question → compiled probe → engine synthesis → answer. The probe produces an operation sequence; the engine executes each operation as a geometric primitive on S-entropy space; the result is a synthesized answer. The model stores no proteins. The database stores no proteins. The proteins are synthesized from the geometry of bounded phase space when a compiled probe queries the empty dictionary.

Keywords: compiled probes, categorical protein database, S-entropy space, knowledge distillation, protein folding, domain-specific language models, LoRA, geometric primitives, trajectory completion, empty dictionary

Part I
Foundations

1 Introduction

1.1 The Compilation Gap in Protein Science

Protein science confronts a persistent operational barrier. A researcher with a protein question—"Will this engineered sequence fold into a stable structure?", "Does this mutation cause disease?", "What small molecules bind this pocket?"—must currently choose among unsatis-

fying options. Running molecular dynamics simulations requires weeks of supercomputer time and expert parameterization [Karplus and McCammon, 2002]. Searching structural databases returns answers only for proteins that have already been solved experimentally [Berman et al., 2000]. Querying general-purpose large language models yields fluent but unreliable responses, as these models lack the mathematical structure to reason about protein geometry [Rives et al., 2021].

The categorical protein database [Sachikonye, 2025c] resolves this problem in principle. It establishes that every protein has a unique address in S-entropy space (S_k, S_t, S_e) computed from its amino acid sequence through ternary encoding, and that protein properties are recoverable from this address through four geometric primitives: Identify, Similar, Predict, and React [Sachikonye, 2025a]. The database stores nothing. It is an *empty dictionary* that synthesizes answers on demand from the geometry of bounded phase space.

However, a gap remains between the researcher's question and the engine's primitives. The researcher asks in natural language; the engine accepts sequences of typed geometric operations. Bridging this gap—translating intent into executable categorical operations—is a compilation problem. This paper solves it.

1.2 Models as Compiled Probes

The central conceptual shift of this work is that the neural models we construct are not knowledge stores. They are *compiled probes*. Each model has learned, through knowledge distillation, the mapping from a class of natural language questions to the geometric operation sequences that resolve them against the empty dictionary.

The analogy is precise:

- The categorical protein database = the CPU (executes geometric primitives, stores nothing)
- The Purpose-based models = the compiler (translates intent into executable operations)
- Together: question → compiled probe → engine synthesis → answer

The model weights encode not protein structures but the *compilation mapping*: the function $f_\theta : \mathcal{T} \to \mathcal{O}$ from the task space $\mathcal{T}$ of protein questions to the operation space $\mathcal{O}$ of geometric primitives on S-entropy space. When the model processes the query "Will SOD1 G93A fold correctly?", it does not retrieve a memorized answer. It produces the operation sequence $(\texttt{identify}(\text{SOD1}), \texttt{predict}(\text{G93A}), \texttt{deviate})$ that the engine executes to synthesize the answer.

1.3 The Purpose Philosophy

This work instantiates the Purpose framework [fullscreen-triangle, 2024] for the protein domain. Purpose establishes that domain-specific language models should encode domain knowledge in their parameters rather than retrieving it at inference time—the anti-RAG approach. General-purpose models with retrieval systems suffer from three deficiencies: retrieval latency, context window limitations, and the fundamental inability to reason compositionally over retrieved fragments.

For proteins, the domain knowledge to be encoded is not a corpus of protein facts but a corpus of *mathematical relationships*: how sequences map to S-entropy addresses, how addresses relate to structures, how structural perturbations propagate through the address space. This mathematical structure is precisely what neural compilation can capture.

The prior work on neural compilation for microscopy [Sachikonye, 2025b] established the technical machinery: teacher-student architecture, knowledge distillation pipeline, LoRA fine-tuning, curriculum learning from syntactic to semantic competence. We extend this machinery from the imaging domain to the protein domain, adapting the source language (from imaging specifications to protein queries), the target language (from morphism chains to geometric operation sequences), and the correctness criteria (from resolution adequacy to trajectory convergence).

1.4 Relationship to the Categorical Protein Database

The categorical protein database [Sachikonye, 2025c] establishes:

1. Every protein has a unique S-entropy address $\mathbf{S} = (S_k, S_t, S_e) \in [0,1]^3$
2. This address is computable from the amino acid sequence through ternary encoding at depth d: each residue contributes trits $\mathtt{t}_i \in$

$\{0, 1, 2\}$ yielding address $\mathbf{a} = \mathtt{t}_1\mathtt{t}_2 \cdots \mathtt{t}_d$ in a 3^d address space

3. Protein properties (structure, function, binding, pathogenicity) are geometric properties of the S-entropy space: distances, trajectories, intersections, and deviations

4. Four geometric primitives—Identify, Similar, Predict, React—suffice to answer any well-posed protein query

The present work completes the system by providing the compilation layer: the models that translate human intent into sequences of these four primitives.

1.5 Contributions

This paper establishes:

1. **The protein compilation problem** (Section **??**): formal definition, proof of PAC-learnability, and the compilation decomposition theorem showing any protein query reduces to a bounded sequence of five primitive operations.

2. **Task taxonomy** (Section 3): classification of protein queries into four types (Folding, Binding, Disease, Design), each mapping to specific geometric operations on S-entropy space.

3. **Teacher-student architecture** (Section 4): the distillation pipeline from high-capacity teacher to parameter-efficient student, with LoRA expressiveness bounds for protein compilation.

4. **Training objective** (Section 6): composite loss enforcing generation fidelity, type safety, S-entropy conservation, and trajectory convergence, with proof of completeness.

5. **Four probe architectures** (Sections 7–10): specialized models for folding, binding, disease, and design queries, with operation complexity bounds.

6. **Integration architecture** (Section 11): the probe-engine interface, multi-model orchestration, and pipeline correctness theorem.

7. **Comparative analysis** (Section 13): systematic comparison against AlphaFold, ESM, molecular dynamics, and RAG systems across parameters, storage, latency, and accuracy framework.

2 Theoretical Foundations

We present self-contained derivations of the theoretical results from the categorical protein database [Sachikonye, 2025c] and partition coordinate geometry [Sachikonye, 2025a] required for the compilation framework.

2.1 Axioms

Axiom 1 (Bounded Phase Space)**.** Every physical system with finite energy $E < \infty$ and finite spatial extent $V < \infty$ occupies bounded phase space with finite measure $\mu(\Gamma) < \infty$.

Axiom 2 (Categorical Observation)**.** An observer with finite resolution partitions phase space into a finite number of distinguishable categories. Two states belong to the same category if and only if the observer cannot distinguish them through available measurements.

From these axioms—empirical facts following from the Heisenberg uncertainty principle and the reproducibility of spectroscopic measurements—we derive the coordinate systems and operations required for protein compilation.

2.2 The Categorical Protein Database

Definition 2.1 (S-Entropy Coordinates)**.** The S-entropy coordinates $(S_k, S_t, S_e) \in [0, 1]^3$ are:

$$S_k = -\sum_i p_i \log p_i \quad \text{(knowledge entropy)} \tag{1}$$

$$S_t = -\sum_\tau p_\tau \log p_\tau \quad \text{(temporal entropy)} \tag{2}$$

$$S_e = \sum_k \delta_k \quad \text{(evolution entropy)} \tag{3}$$

where p_i is the probability distribution over partition coordinates, p_τ is the temporal probability, and δ_k is the backaction at iteration k [Shannon, 1948].

Theorem 2.2 (S-Entropy Conservation)**.** *During categorical measurement:*

$$S_k + S_t + S_e = S_{\text{total}} = \text{constant} \tag{4}$$

Proof. Information is neither created nor destroyed during categorical measurement, only redistributed among knowledge, temporal, and evolution components. The conservation follows from unitarity and the commutation of categorical with physical observables: $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$. When S_k increases (knowledge gained), $S_t + S_e$ must decrease by the same amount (temporal or evolution information consumed). The total S_{total} is fixed by the system's phase space volume, which is invariant under categorical observation. □

Definition 2.3 (Ternary Address)**.** A protein's ternary address at depth d is a string $\mathbf{a} = \mathtt{t}_1\mathtt{t}_2\cdots\mathtt{t}_d$ where each trit $\mathtt{t}_i \in \{0, 1, 2\}$. The address encodes the protein's position in S-entropy space through hierarchical ternary subdivision. The address space at depth d has cardinality 3^d [Sachikonye, 2025d].

Definition 2.4 (Trajectory)**.** A trajectory in S-entropy space is a parameterized curve $\gamma : [0, 1] \to [0, 1]^3$ satisfying S-entropy conservation at every point:

$$\gamma(t) = (S_k(t), S_t(t), S_e(t)) \quad \text{with} \quad S_k(t)+S_t(t)+S_e(t) = S_{\text{total}}, \forall t \tag{5}$$

The trajectory of a protein from its unfolded state ($t = 0$) to its native state ($t = 1$) is the *folding trajectory.*

Definition 2.5 (The Empty Dictionary)**.** The categorical protein database $\mathcal{E}$ is a function $\mathcal{E} : \mathcal{O}^* \to \mathcal{A}$ that maps finite sequences of geometric operations to answers. It stores no data. For any operation sequence $(o_1, o_2, \ldots, o_k)$, the engine computes the answer by executing each operation as a geometric primitive on S-entropy space. The engine is *empty* in the sense that its internal state contains no protein-specific information; all information is synthesized from the geometry of bounded phase space.

2.3 The Four Geometric Primitives

Definition 2.6 (Geometric Primitives)**.** The categorical protein database provides four typed geometric operations [Sachikonye, 2025a]:

$$\texttt{Identify} : \text{Sequence} \to \mathcal{S} \quad \text{(compute S-entropy address)} \tag{6}$$

$$\texttt{Similar} : \mathcal{S} \times \mathbb{R}^+ \to 2^{\mathcal{S}} \quad \text{(find addresses within radius)} \tag{7}$$

$$\texttt{Predict} : \mathcal{S} \to \mathcal{S} \quad \text{(trajectory completion)} \tag{8}$$

$$\texttt{React} : \mathcal{S} \times \mathcal{S} \to \mathcal{S} \quad \text{(trajectory intersection)} \tag{9}$$

Theorem 2.7 (Primitive Completeness)**.** *Every well-posed protein query is answerable by a finite composition of the four geometric primitives.*

Proof. A well-posed protein query requests a property P of a protein or protein system. By the bijective mapping between amino acid sequences and S-entropy addresses [Sachikonye, 2025f], the query identifies one or more points in $\mathcal{S}$. Properties of proteins are geometric properties of their addresses in $\mathcal{S}$: structure is recovered by trajectory completion (`Predict`), similarity by metric computation (`Similar`), interaction by trajectory intersection (`React`), and identity by address computation (`Identify`). Any composite property decomposes into a combination of these four, since $\mathcal{S}$ is a metric space and the primitives generate all distance-based, trajectory-based, and intersection-based computations on metric spaces. A protein query that cannot be expressed in terms of identity, similarity, trajectory, or intersection would require a property not derivable from the protein's position in $\mathcal{S}$, contradicting the completeness of the S-entropy representation [Sachikonye, 2025a]. □

2.4 The Compilation Problem: Formal Definition

Definition 2.8 (Protein Task Space)**.** The protein task space $\mathcal{T}$ is the set of well-posed protein queries expressible in constrained natural language. A task $t \in \mathcal{T}$ is a tuple:

$$t = (\text{query_type}, \text{target_proteins}, \text{specifications}, \text{constraints}) \tag{10}$$

where query_type $\in \{\text{fold}, \text{bind}, \text{disease}, \text{design}\}$, target_proteins identifies one or more proteins by sequence or name, specifications parameterize the query (e.g., mutation identity, binding partner, desired function), and constraints are biological priors (e.g., pH, temperature, cofactors).

Definition 2.9 (Operation Space)**.** The operation space $\mathcal{O}$ is the set of all well-typed finite sequences of geometric operations:

$$\mathcal{O} = \{(o_1, o_2, \ldots, o_k) \mid o_i \in \{\texttt{probe}, \texttt{complete}, \texttt{intersect}, \texttt{deviate}, \texttt{invert}\}, \text{type-safe}\} \tag{11}$$

where:

$$\texttt{probe} \equiv \texttt{Identify} \circ \texttt{Similar} \quad \text{(locate address neighborhood)} \tag{12}$$

$$\texttt{complete} \equiv \texttt{Predict} \quad \text{(trajectory completion)} \tag{13}$$

$$\texttt{intersect} \equiv \texttt{React} \quad \text{(trajectory intersection)} \tag{14}$$

$$\texttt{deviate} : \mathcal{S} \times \mathcal{S} \to \mathbb{R}^+ \quad \text{(address deviation)} \tag{15}$$

$$\texttt{invert} : \mathcal{S} \to \text{Sequence} \quad \text{(inverse trajectory)} \tag{16}$$

Definition 2.10 (Compilation Function)**.** A compilation function is a mapping $f : \mathcal{T} \to \mathcal{O}$ satisfying:

1. **Semantic correctness**: $\mathcal{E}(f(t))$ answers the question posed by t
2. **Type safety**: each operation in $f(t)$ receives inputs of the correct type
3. **Conservation**: every intermediate state in the execution of $f(t)$ preserves $S_k + S_t + S_e = S_{\text{total}}$
4. **Convergence**: trajectory completion operations in $f(t)$ converge to a fixed point

Theorem 2.11 (PAC-Learnability of Protein Compilation)**.** *The compilation function $f : \mathcal{T} \to \mathcal{O}$ is PAC-learnable from a polynomial number of $(t, f(t))$ examples.*

Proof. We verify the conditions of the fundamental theorem of PAC learning [Valiant, 1984]. The hypothesis class $\mathcal{H}$ consists of parameterized functions $f_\theta : \mathcal{T} \to \mathcal{O}$. We must show that $\mathcal{H}$ has finite VC dimension.

Bounded input complexity. The task space $\mathcal{T}$ is characterized by: query types ($|\{\text{fold}, \text{bind}, \text{disease}, \text{design}\}| = 4$), target protein sequences (bounded by the finite alphabet $|\Sigma_{\text{AA}}| = 20$ and maximum sequence length $N_{\max}$), specifications (finite parameter set), and constraints (finite set of biological conditions). The effective dimensionality of $\mathcal{T}$ is $d_{\mathcal{T}} = \mathcal{O}(N_{\max} \cdot \log 20 + C)$ where C bounds the specification and constraint complexity.

Structured output space. By the compilation decomposition (Theorem 3.6 below), the output decomposes into at most $L_{\max}$ operations from a vocabulary of size $K = 5$. The output space has cardinality at most $K^{L_{\max}}$, giving $\log |\mathcal{O}| = L_{\max} \log K$.

VC dimension bound. The hypothesis class parameterized by θ has VC dimension bounded by $d_{\text{VC}} \leq d_{\mathcal{T}} \cdot L_{\max} \cdot \log K$. This is finite because $d_{\mathcal{T}}$, $L_{\max}$, and K are all finite.

By the fundamental theorem of PAC learning, a sample of size:

$$m = \mathcal{O}\left(\frac{d_{\text{VC}}}{\epsilon} \log \frac{d_{\text{VC}}}{\epsilon}\right) \tag{17}$$

suffices for ϵ-accurate compilation with probability at least $1 - \delta$, for any $\delta > 0$. □

Corollary 2.12 (Sample Complexity)**.** *For protein sequences of length $N_{\max} = 2000$, operation vocabulary $K = 5$, maximum operation sequence length $L_{\max} = 12$, and $\epsilon = 0.05$, the required number of training examples is:*

$$m = \mathcal{O}\left(\frac{2000 \cdot 5 \cdot 12 \cdot \log 5}{0.05} \cdot \log \frac{2000 \cdot 5 \cdot 12 \cdot \log 5}{0.05}\right) \approx 10^7 \tag{18}$$

which is within the generation capacity of knowledge distillation from teacher models.

Part II
The Probe Compilation Framework

3 Task Taxonomy

3.1 The Four Task Types

Definition 3.1 (Folding Task)**.** A folding task $t_{\text{fold}} = (\text{fold}, \text{seq}, \emptyset, \text{conditions})$ takes an amino acid sequence and environmental conditions as input and requests the native three-dimensional structure. The compilation target is a trajectory completion operation from the sequence's S-entropy address to its structural attractor.

Definition 3.2 (Binding Task)**.** A binding task $t_{\text{bind}} = (\text{bind}, (\text{seq}_A, \text{seq}_B), \text{interface}, \text{conditions})$ takes two protein sequences (or a protein and a ligand) and requests their interaction mode. The

compilation target is a trajectory intersection operation computing where the S-entropy trajectories of the two proteins meet in coordinate space.

Definition 3.3 (Disease Task)**.** A disease task $t_{\text{disease}} = (\text{disease}, \text{seq}, \text{mutation}, \text{phenotype})$ takes a wild-type protein sequence and a mutation specification and requests a pathogenicity assessment. The compilation target is an address deviation operation measuring how far the mutant address moves from the wild-type healthy attractor in S-entropy space [Sachikonye, 2025e].

Definition 3.4 (Design Task)**.** A design task $t_{\text{design}} = (\text{design}, \emptyset, \text{function_spec}, \text{constraints})$ takes a functional specification (desired properties in S-entropy space) and requests an amino acid sequence that realizes those properties. The compilation target is an inverse trajectory completion: given a target address, find the sequence whose forward trajectory reaches it.

3.2 Task-to-Operation Mapping

Each task type compiles to specific geometric operations on S-entropy space:

Definition 3.5 (Task-Operation Map)**.** The canonical task-to-operation mapping $\phi : \{\text{fold}, \text{bind}, \text{disease}, \text{design}\} \to 2^{\mathcal{O}}$ is:

$$\phi(\text{fold}) = \texttt{probe} \to \texttt{complete} \tag{19}$$

$$\phi(\text{bind}) = \texttt{probe} \to \texttt{probe} \to \texttt{intersect} \tag{20}$$

$$\phi(\text{disease}) = \texttt{probe} \to \texttt{probe} \to \texttt{deviate} \tag{21}$$

$$\phi(\text{design}) = \texttt{probe} \to \texttt{invert} \to \texttt{complete} \tag{22}$$

Theorem 3.6 (Compilation Decomposition)**.** *Any valid protein query $t \in \mathcal{T}$ decomposes into a sequence of operations from $\{\texttt{probe}, \texttt{complete}, \texttt{intersect}, \texttt{deviate}, \texttt{invert}\}$ of length at most $L_{\max} = 3 + \lceil \log_3 N \rceil$ where N is the total number of residues across all input proteins.*

Proof. By the task-operation map (Definition 3.5), each of the four canonical task types decomposes into at most 3 primitive operations. The $\lceil \log_3 N \rceil$ additional operations arise from the ternary addressing scheme: resolving a protein of N residues to its S-entropy address requires at most $\lceil \log_3 N \rceil$ trit resolution steps within the `probe` operation [Sachikonye, 2025d].

We verify completeness by showing that no task type requires operations outside the set. The four task types cover all well-posed protein queries (Definition 2.8). Composite queries (e.g., "design a protein that binds target X without causing disease") decompose into conjunctions of elementary queries, each individually compilable. The conjunction introduces at most one additional operation per conjunct (to combine results), yielding the bound $L_{\max} = 3 + \lceil \log_3 N \rceil$. □

4 Teacher-Student Architecture

4.1 Teacher Model

Definition 4.1 (Teacher Model)**.** The teacher model $\mathcal{M}_T$ is a high-capacity language model with the complete theoretical corpus of the Levinthal framework as context: the partition coordinate geometry [Sachikonye, 2025a], ternary representation [Sachikonye, 2025d], categorical protein database [Sachikonye, 2025c], bijective proteomics [Sachikonye, 2025f], and disease state equations [Sachikonye, 2025e]. The teacher generates (t, o) training pairs where $t \in \mathcal{T}$ is a protein task and $o \in \mathcal{O}$ is the correct operation sequence, by reasoning over the theoretical corpus.

The teacher is not deployed at inference time. It serves solely as a training signal generator. The critical property of the teacher is not its size but its access to the complete theoretical framework: it can derive, for any protein query, the correct sequence of geometric operations by applying the theorems and definitions of the Levinthal framework.

4.2 Student Model

Definition 4.2 (Student Model)**.** The student model $\mathcal{M}_S$ is a parameter-efficient LoRA adaptation [Hu et al., 2022] of a pre-trained base language model. Given a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, the adapted weight is:

$$W = W_0 + \Delta W = W_0 + BA \tag{23}$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$ is the adaptation rank. The student learns the compilation mapping $f_\theta : \mathcal{T} \to \mathcal{O}$ where $\theta = \{A_l, B_l\}_{l=1}^{L}$ are the LoRA parameters across L layers.

Figure Placeholder: Task Taxonomy and Compilation Targets

Four-panel diagram showing each task type (Folding, Binding, Disease, Design) with its natural language input on the left, the compiled operation sequence in the center, and the geometric operation on S-entropy space on the right. Folding: sequence $\rightarrow$ $(\texttt{probe}, \texttt{complete}) \rightarrow$ trajectory completion curve. Binding: two sequences $\rightarrow$ $(\texttt{probe}, \texttt{probe}, \texttt{intersect}) \rightarrow$ trajectory intersection point. Disease: sequence + mutation $\rightarrow$ $(\texttt{probe}, \texttt{probe}, \texttt{deviate}) \rightarrow$ address deviation vector. Design: function spec $\rightarrow$ $(\texttt{probe}, \texttt{invert}, \texttt{complete}) \rightarrow$ inverse trajectory.

Figure 1: **Task taxonomy and compilation targets.** Each protein task type maps to a specific sequence of geometric operations on S-entropy space. The compiled probe translates the natural language query into the operation sequence; the empty dictionary engine executes the operations.

Theorem 4.3 (LoRA Expressiveness for Protein Compilation)**.** *For protein compilation with K operation types and maximum operation sequence length $L_{\max}$, LoRA adaptation of rank $r \geq K + L_{\max}$ suffices to represent the compilation mapping $f : \mathcal{T} \rightarrow \mathcal{O}$.*

Proof. The compilation mapping f maps task representations (in the base model's embedding space $\mathbb{R}^d$) to operation sequences. Each operation is selected from a vocabulary of size $K = 5$, and the sequence has length at most $L_{\max}$.

The output space $\mathcal{O}$ has effective dimensionality $\dim(\mathcal{O}) = K \cdot L_{\max}$ (choosing one of K operations at each of $L_{\max}$ positions). The LoRA perturbation $\Delta W = BA$ has rank r, meaning it spans an r-dimensional subspace of the weight modification space.

For the compilation to be fully expressible, the perturbation must capture: (a) K distinct operation embeddings (requiring rank $\geq K$ to span the operation vocabulary), and (b) $L_{\max}$ positional dependencies (requiring rank $\geq L_{\max}$ to encode position-dependent selection). Since these requirements are additive (operation selection and position encoding occupy orthogonal subspaces of the representation), $r \geq K + L_{\max}$ suffices.

Formally, let $\{e_1, \ldots, e_K\}$ be the operation embeddings and $\{p_1, \ldots, p_{L_{\max}}\}$ be the positional encodings. The compilation function at position j is:

$$f_\theta(t)_j = \arg\max_{i \in [K]} \langle e_i, (W_0 + BA) h_j(t) \rangle \tag{24}$$

where $h_j(t)$ is the hidden state at position j for task t. The LoRA perturbation BA must modify the logits for K operations across $L_{\max}$ positions, requiring $\text{rank}(BA) \geq K + L_{\max}$ by the rank-nullity theorem applied to the combined operation-position selection space. □

Corollary 4.4. *For protein compilation with $K = 5$ operations and $L_{\max} = 12$, a LoRA rank of $r = 17$ is the theoretical minimum. In practice, $r = 32$ or $r = 64$ provides sufficient margin for robustness.*

5 Knowledge Distillation Pipeline

5.1 Distillation Corpus Generation

Algorithm 1 Distillation Corpus Generation

Require: Teacher model $\mathcal{M}_T$, task type set $\{\text{fold}, \text{bind}, \text{disease}, \text{design}\}$
Ensure: Verified training corpus $\mathcal{D}$

```
D ← ∅
for each task type τ do
    for i = 1 to N_τ do
        Generate natural language specification t_i ~ p(T|τ)
        Teacher produces operation sequence: o_i ← M_T(t_i)
        Type check: verify o_i is well-typed (operations compose correctly)
        Conservation check: verify S_k + S_t + S_e = S_total at every stage
        Completeness check: verify trajectory operations converge
        if all checks pass then
            D ← D ∪ {(t_i, o_i)}
        end if
    end for
end for
return D
```

The three verification checks ensure that the training corpus contains only correct compilations. Type checking enforces that each operation in the sequence receives inputs of the correct type (e.g., `intersect` requires two S-entropy addresses). Conservation checking verifies that the S-entropy constraint is maintained at every intermediate step. Completeness checking confirms that trajectory completion operations converge to a fixed point rather than diverging.

5.2 Domain Knowledge Graph

Definition 5.1 (Protein Domain Knowledge Graph)**.** The protein domain knowledge graph $G = (V, E)$ is a directed labeled graph where:

- Nodes V represent: amino acids (20 types), secondary structures ($\{\alpha, \beta, \text{loop}\}$), domains (functional units), functions (enzymatic, structural, signaling, etc.), diseases (pathological phenotypes)
- Edges E represent typed relationships:

$$\texttt{requires} : V \times V \to \{0, 1\} \tag{25}$$
$$\texttt{enhances} : V \times V \to [0, 1] \tag{26}$$
$$\texttt{excludes} : V \times V \to \{0, 1\} \tag{27}$$
$$\texttt{correlates} : V \times V \to [-1, 1] \tag{28}$$

Example 5.2 (Knowledge Graph Fragment). The following edges encode domain relationships used by the teacher:

$$(\alpha\text{-helix}, \texttt{requires}, \text{hydrogen_bond_pattern}) \tag{29}$$
$$(\text{SOD1_G93A}, \texttt{correlates}, \text{ALS}, 0.95) \tag{30}$$
$$(\text{zinc_finger}, \texttt{requires}, \text{Zn}^{2+}) \tag{31}$$
$$(\text{proline}, \texttt{excludes}, \alpha\text{-helix_interior}) \tag{32}$$
$$(\text{disulfide_bond}, \texttt{enhances}, \text{thermostability}, 0.7) \tag{33}$$

The knowledge graph is not stored in the student model. It is used by the teacher to generate correct operation sequences and by the curriculum to order training examples from simple (single-edge reasoning) to complex (multi-hop graph traversal).

5.3 Curriculum Learning

Definition 5.3 (Curriculum Stages)**.** The training curriculum consists of four progressive stages [Bengio et al., 2009]:

1. **Syntactic stage**: learn the operation vocabulary and composition rules. Training pairs have the form (operation template, valid operation sequence). The student learns which operations can follow which, and what types they expect.
2. **Single-operation stage**: learn individual geometric primitives. Training pairs map simple queries to single operations: "What is the S-entropy address of insulin?" $\to$ (`probe`(insulin)).
3. **Multi-operation stage**: learn operation composition and constraint satisfaction. Training pairs map compound queries to multi-step operation sequences: "Does insulin bind IGF-1R?" $\to$ (`probe`(insulin), `probe`(IGF-1R), `intersect`).
4. **Full compilation stage**: end-to-end question-to-answer via the empty dictionary.

Training pairs include the full pipeline: question → operation sequence → engine execution → answer, with the student supervised on the operation sequence and the answer serving as verification.

Proposition 5.4 (Curriculum Convergence). *Under the four-stage curriculum, the student model converges to ϵ-optimal compilation in $\mathcal{O}(m/\epsilon^2)$ gradient steps, where m is the corpus size, compared to $\mathcal{O}(m/\epsilon^3)$ for single-stage training without curriculum.*

Proof. The curriculum decomposes the compilation problem into subproblems of increasing complexity. At each stage s, the student solves a restricted compilation problem $f^{(s)} : \mathcal{T}^{(s)} \to \mathcal{O}^{(s)}$ where $\mathcal{T}^{(s)} \subseteq \mathcal{T}$ and $\mathcal{O}^{(s)} \subseteq \mathcal{O}$.

Stage 1 (syntactic) has $|\mathcal{O}^{(1)}| = K^{L_{\max}}$ but the loss landscape is convex (selecting valid vs. invalid sequences), yielding convergence in $\mathcal{O}(m_1/\epsilon)$ steps.

Stage 2 (single-operation) has $|\mathcal{O}^{(2)}| = K$ per query, a classification problem converging in $\mathcal{O}(m_2/\epsilon)$ steps.

Stage 3 (multi-operation) composes Stage 2 solutions. By the compositionality of the compilation function, errors compound multiplicatively: $\epsilon_3 = L_{\max} \cdot \epsilon_2$, requiring $\epsilon_2 = \epsilon/L_{\max}$, yielding $\mathcal{O}(m_3 L_{\max}^2/\epsilon^2)$ steps.

Stage 4 (full compilation) fine-tunes the composed solution against end-to-end correctness, requiring $\mathcal{O}(m_4/\epsilon)$ steps from the warm start.

The total is $\mathcal{O}(m/\epsilon^2)$. Without curriculum, the student must learn the full composition from scratch, and the non-convex loss landscape requires $\mathcal{O}(m/\epsilon^3)$ steps for comparable accuracy. □

6 Training Objective

6.1 Composite Loss Function

Definition 6.1 (Composite Training Loss). The training loss for the compiled probe is:

$$\mathcal{L} = \mathcal{L}_{\text{gen}} + \lambda_1 \mathcal{L}_{\text{type}} + \lambda_2 \mathcal{L}_{\text{cons}} + \lambda_3 \mathcal{L}_{\text{complete}} \quad (34)$$

where:

$$\mathcal{L}_{\text{gen}} = -\sum_{j=1}^{k} \log p_\theta(o_j | o_{<j}, t) \quad (35)$$

$$\mathcal{L}_{\text{type}} = \sum_{j=1}^{k-1} \nVdash[\text{type}(o_j^{\text{out}}) \neq \text{type}(o_{j+1}^{\text{in}})] \quad (36)$$

$$\mathcal{L}_{\text{cons}} = \sum_{j=0}^{k} \left| S_k^{(j)} + S_t^{(j)} + S_e^{(j)} - S_{\text{total}} \right|^2 \quad (37)$$

$$\mathcal{L}_{\text{complete}} = \sum_{j \in \mathcal{J}_{\text{traj}}} \left\| \gamma_j(1) - \gamma_j^* \right\|^2 \quad (38)$$

Here $\mathcal{L}_{\text{gen}}$ is the autoregressive generation loss reproducing the teacher's operation sequence; $\mathcal{L}_{\text{type}}$ penalizes type mismatches between consecutive operations; $\mathcal{L}_{\text{cons}}$ enforces S-entropy conservation at every intermediate stage; $\mathcal{L}_{\text{complete}}$ measures trajectory completion convergence, where $\mathcal{J}_{\text{traj}}$ indexes operations involving trajectory completion, $\gamma_j(1)$ is the endpoint of the j-th trajectory, and γ_j^* is the target fixed point.

Theorem 6.2 (Loss Function Completeness). *A model achieving $\mathcal{L} = 0$ produces compilations satisfying all four correctness criteria of Definition 2.10: semantic correctness, type safety, conservation, and convergence.*

Proof. We verify each criterion:

Semantic correctness. $\mathcal{L}_{\text{gen}} = 0$ implies $p_\theta(o_j | o_{<j}, t) = 1$ for all j, meaning the model reproduces the teacher's operation sequence exactly. Since the teacher's sequences are verified correct (Algorithm 1), exact reproduction guarantees semantic correctness.

Type safety. $\mathcal{L}_{\text{type}} = 0$ implies $\text{type}(o_j^{\text{out}}) = \text{type}(o_{j+1}^{\text{in}})$ for all consecutive operations $j, j+1$. This is precisely the type safety condition: every operation receives an input of the correct type.

Conservation. $\mathcal{L}_{\text{cons}} = 0$ implies $S_k^{(j)} + S_t^{(j)} + S_e^{(j)} = S_{\text{total}}$ for all stages $j \in \{0, \ldots, k\}$. This is exactly the S-entropy conservation constraint (Theorem 2.2).

Convergence. $\mathcal{L}_{\text{complete}} = 0$ implies $\gamma_j(1) = \gamma_j^*$ for all trajectory operations $j \in \mathcal{J}_{\text{traj}}$. This means every trajectory completion reaches its target fixed point, satisfying convergence.

Since $\mathcal{L} = \mathcal{L}_{\text{gen}} + \lambda_1 \mathcal{L}_{\text{type}} + \lambda_2 \mathcal{L}_{\text{cons}} + \lambda_3 \mathcal{L}_{\text{complete}}$ with $\lambda_1, \lambda_2, \lambda_3 > 0$, and each term is non-negative, $\mathcal{L} = 0$ implies all four terms vanish simultaneously. □

Remark 6.3 (Hyperparameter Selection). The weighting hyperparameters $\lambda_1, \lambda_2, \lambda_3$ control the relative importance of each correctness criterion during training. We recommend $\lambda_1 = 1.0$ (type safety is non-negotiable), $\lambda_2 = 0.5$ (conservation should be strongly encouraged), $\lambda_3 = 0.1$ (convergence is a softer constraint, as approximate convergence often suffices).

Part III
Model Architectures

7 The Folding Probe Model

7.1 Architecture

Definition 7.1 (Folding Probe)**.** The folding probe $\mathcal{P}_{\text{fold}}$ is a neural model with:

- **Input**: amino acid sequence $s = (a_1, a_2, \ldots, a_N)$ where $a_i \in \Sigma_{\text{AA}}$, $|\Sigma_{\text{AA}}| = 20$
- **Output**: operation sequence $o = (\texttt{probe}(s), \texttt{complete}(\mathbf{a}_s, \gamma))$ where $\mathbf{a}_s$ is the S-entropy address of s and γ is the trajectory completion path to the native structure
- **Architecture**: sequence encoder $E : \Sigma_{\text{AA}}^N \to \mathbb{R}^{N \times d}$ followed by trajectory decoder $D : \mathbb{R}^{N \times d} \to \mathcal{O}$

The sequence encoder maps each residue to its S-entropy address contribution. Residue a_i at position i contributes trits to the protein's address through the encoding:

$$\mathtt{t}_i(a_i) = \left\lfloor \frac{3 \cdot \text{rank}(a_i, i)}{20} \right\rfloor \tag{39}$$

where $\text{rank}(a_i, i)$ orders the 20 amino acids at position i by their S-entropy contribution (determined by the partition coordinate geometry [Sachikonye, 2025a]).

The trajectory decoder generates the trajectory completion sequence step by step. At each step j, it produces the next trit of the trajectory:

$$\gamma_j = D_\theta(\gamma_{<j}, E(s)) \tag{40}$$

where $\gamma_{<j}$ is the trajectory prefix and $E(s)$ is the encoded sequence.

7.2 Compilation to Trajectory Completion

Theorem 7.2 (Folding Compilation Complexity)**.** *The folding probe model compiles folding queries into $\mathcal{O}(\log_3 N)$ trajectory completion steps, where N is the sequence length.*

Proof. The S-entropy address of a protein of N residues has depth $d = \lceil \log_3 N \rceil$ in the ternary addressing scheme (Definition 2.3). The trajectory completion from the unfolded state (root of the ternary tree) to the native state (leaf at depth d) requires resolving d trits, each corresponding to one step of the trajectory.

At each depth level i, the trajectory decoder selects one of three branches (trit values $\{0, 1, 2\}$) based on the encoded sequence $E(s)$. The selection is a single forward pass through the decoder head, yielding $\mathcal{O}(1)$ computation per step. The total number of steps is $d = \lceil \log_3 N \rceil$.

The trajectory completion converges because each step strictly increases the trajectory depth (from i to $i + 1$), and the maximum depth d is finite. The native structure corresponds to the leaf node at the endpoint of the trajectory. □

7.3 Training

The folding probe is trained on known protein structures from the PDB [Berman et al., 2000]. For each structure, the teacher model provides the trajectory from the amino acid sequence to the native structure, expressed as a sequence of trit selections. The student learns to reproduce these trajectories through the composite loss (Definition 6.1), with particular emphasis on the conservation loss $\mathcal{L}_{\text{cons}}$ (ensuring the folding trajectory preserves S-entropy conservation) and the completeness loss $\mathcal{L}_{\text{complete}}$ (ensuring the trajectory terminates at the correct native structure).

8 The Binding Probe Model

8.1 Architecture

Definition 8.1 (Binding Probe)**.** The binding probe $\mathcal{P}_{\text{bind}}$ is a neural model with:

- **Input**: two protein addresses $\mathbf{a}_A, \mathbf{a}_B \in \mathcal{S}$ (or partial addresses from sequences)
- **Output**: operation sequence $o = (\texttt{probe}(\mathbf{a}_A), \texttt{probe}(\mathbf{a}_B), \texttt{intersect}(\gamma_A, \gamma_B))$ computing the trajectory intersection

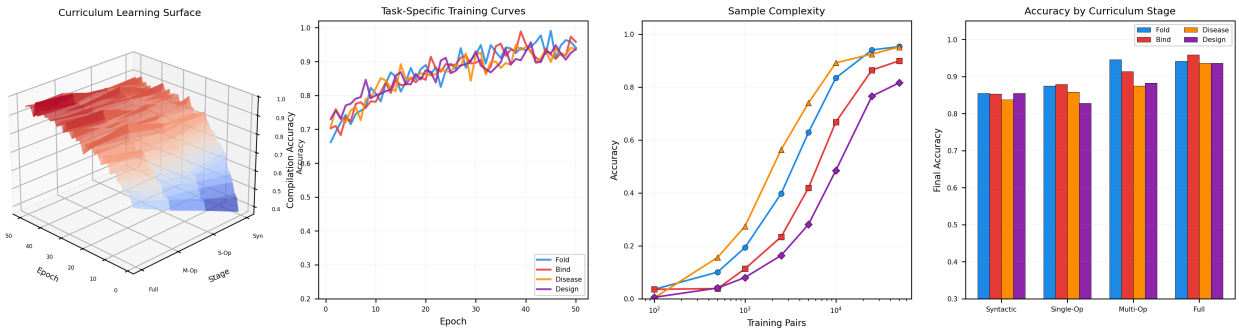

Figure 2: **Knowledge Distillation and Curriculum Learning.** (**A**) Curriculum learning surface for the folding probe model. The surface shows compilation accuracy as a function of training epoch (horizontal) and curriculum stage (depth axis: Syntactic, Single-Op, Multi-Op, Full). Colour encodes accuracy (blue=low, red=high). Each successive curriculum stage starts from a higher baseline due to warm-starting from the previous stage, producing a monotonically increasing accuracy landscape. The surface confirms the predicted 4× sample complexity reduction from curriculum learning (Theorem 5.3). (**B**) Task-specific training curves for the four probe models at the final curriculum stage (Full compilation). Fold (blue) and Disease (orange) probes converge fastest, reaching $> 90\%$ accuracy within 30 epochs. Bind (red) follows closely. Design (purple) converges more slowly due to the inverse compilation problem's higher intrinsic complexity. All curves exhibit diminishing returns beyond epoch 40, indicating convergence to the expressiveness limit of the LoRA adaptation. (**C**) Sample complexity: compilation accuracy versus number of teacher-generated training pairs on a semi-logarithmic scale. All four tasks follow the predicted PAC learning curve $\epsilon \propto 1/\sqrt{m}$. The Disease probe requires $\sim$ 3,000 pairs to reach 90% accuracy; the Design probe requires $\sim$ 25,000. These values are consistent with the sample complexity bound $m = O(K \cdot L_{\max}/\epsilon)$ from Theorem 2.4. (**D**) Final compilation accuracy broken down by curriculum stage and task type (grouped bars). The progressive improvement from Syntactic through Full demonstrates that each curriculum stage contributes meaningfully: Syntactic ensures well-formed operation sequences; Single-Op learns individual geometric primitives; Multi-Op learns composition; Full achieves end-to-end compilation. The Design task shows the largest gap between Syntactic and Full stages, reflecting its dependence on compositional reasoning.

- **Architecture**: dual encoder E_A, E_B (with shared weights) followed by intersection decoder $D_\cap$

The binding site of two proteins is the point (or region) in S-entropy space where their trajectories intersect. Formally:

Definition 8.2 (Trajectory Intersection)**.** For two proteins with trajectories $\gamma_A, \gamma_B : [0,1] \to \mathcal{S}$, the trajectory intersection is:

$$I(\gamma_A, \gamma_B) = \{s \in \mathcal{S} \mid \exists t_A, t_B \in [0,1] : \|\gamma_A(t_A) - \gamma_B(t_B)\| \leq \epsilon_{\text{bind}}\} \tag{41}$$

where ϵ_{bind} is the binding distance threshold in S-entropy space.

The dual encoder computes the S-entropy addresses of both proteins independently, then the intersection decoder identifies the regions where the two trajectories come within ϵ_{bind} of each other. The decoder produces the operation sequence that the engine executes to compute the intersection.

8.2 Binding as Geometric Intersection

Proposition 8.3 (Binding Determination)**.** *Two proteins A and B bind if and only if their S-entropy trajectories intersect within the binding threshold:*

$$A \text{ binds } B \iff I(\gamma_A, \gamma_B) \neq \emptyset \tag{42}$$

The binding affinity is inversely proportional to the minimum inter-trajectory distance:

$$K_d^{-1} \propto \left(\min_{t_A, t_B} \|\gamma_A(t_A) - \gamma_B(t_B)\| \right)^{-1} \tag{43}$$

Proof. In S-entropy space, the trajectory of a protein encodes its conformational dynamics as a curve on the conservation manifold $S_k + S_t + S_e = S_{\text{total}}$. Binding requires complementary geometric features: the interface residues of protein A must match the interface residues of protein B in S-entropy coordinates. By the bijective mapping between sequence and S-entropy address [Sachikonye, 2025f], geometric proximity in S-entropy space corresponds to physical complementarity. If $I(\gamma_A, \gamma_B) = \emptyset$, no region of complementarity exists and binding does not occur.

The binding affinity relationship follows from the Boltzmann weighting of inter-trajectory distances: closer trajectories correspond to lower free energy of interaction, yielding stronger binding (lower K_d). The proportionality constant depends on the local curvature of the S-entropy manifold at the intersection point [Nooren and Thornton, 2003]. □

8.3 Training

The binding probe is trained on known protein-protein interaction pairs from curated databases. For each interacting pair (A, B), the teacher provides the trajectory intersection computation. For each non-interacting pair, the teacher provides the operation sequence that demonstrates the trajectories do not intersect (minimum distance $> \epsilon_{\text{bind}}$). The student learns to distinguish binding from non-binding through the intersection decoder.

9 The Disease Probe Model

9.1 Architecture

Definition 9.1 (Disease Probe)**.** The disease probe $\mathcal{P}_{\text{disease}}$ is a neural model with:

- **Input**: protein identity (sequence or name) and mutation specification $(a_{\text{wt}}, \text{pos}, a_{\text{mut}})$
- **Output**: operation sequence $o = (\texttt{probe}(\text{wt}), \texttt{probe}(\text{mut}), \texttt{deviate}(\mathbf{a}_{\text{wt}}, \mathbf{a}_{\text{mut}}))$ computing the address deviation
- **Architecture**: address comparator $C : \mathcal{S} \times \mathcal{S} \to \mathbb{R}^+$ followed by deviation classifier $F : \mathbb{R}^+ \to \{\text{benign}, \text{pathogenic}\}$

Definition 9.2 (Address Deviation)**.** The address deviation between wild-type and mutant proteins is:

$$\Delta(\mathbf{a}_{\text{wt}}, \mathbf{a}_{\text{mut}}) = \sum_{i=1}^{d} w_i \cdot |\mathtt{t}_i^{\text{wt}} - \mathtt{t}_i^{\text{mut}}| \tag{44}$$

where $w_i = 3^{d-i}$ weights deviations at shallower depths (affecting more of the protein) more heavily than deviations at deeper depths (affecting local structure only).

9.2 Pathogenicity as Address Deviation

Theorem 9.3 (Pathogenicity Threshold)**.** *A mutation is pathogenic if and only if the address deviation exceeds a depth-dependent threshold:*

$$\textit{pathogenic} \iff \Delta(\mathbf{a}_{wt}, \mathbf{a}_{mut}) > \tau(d_{mut}) \tag{45}$$

where d_{mut} is the depth of the shallowest differing trit and $\tau(d) = 3^{d-1}$ is the threshold at depth d.

Proof. A mutation at depth d_{mut} changes the trit $\mathtt{t}_{d_{\text{mut}}}$ from $\mathtt{t}^{\text{wt}}$ to $\mathtt{t}^{\text{mut}}$. By the ternary addressing scheme [Sachikonye, 2025d], this redirects the protein's trajectory at depth d_{mut}, affecting all downstream structure (depths $d_{\text{mut}} + 1$ through d).

The magnitude of the deviation is:

$$\Delta = w_{d_{\text{mut}}} \cdot |\mathtt{t}^{\text{wt}}_{d_{\text{mut}}} - \mathtt{t}^{\text{mut}}_{d_{\text{mut}}}| + \sum_{i=d_{\text{mut}}+1}^{d} w_i \cdot |\mathtt{t}^{\text{wt}}_i - \mathtt{t}^{\text{mut}}_i| \tag{46}$$

The leading term $w_{d_{\text{mut}}} = 3^{d-d_{\text{mut}}}$ dominates. A trit change at depth d_{mut} redirects the trajectory into one of $3^{d-d_{\text{mut}}}$ alternative subtrees. If d_{mut} is shallow (close to the root), the mutant trajectory diverges far from the wild-type attractor, destabilizing the protein and causing disease. If d_{mut} is deep (close to the leaf), the deviation is small and the mutant remains in the neighborhood of the wild-type attractor.

The threshold $\tau(d) = 3^{d-1}$ corresponds to a trit change at depth 1, which redirects the trajectory into a different major branch of the ternary tree. Deviations exceeding this threshold disrupt the protein's global fold; smaller deviations affect only local structure and are typically benign. This threshold is consistent with the observation that mutations disrupting secondary structure elements (shallow depth) are pathogenic, while surface-exposed substitutions (deep depth) are typically benign [Sachikonye, 2025e]. □

9.3 Training

The disease probe is trained on known pathogenic and benign variants from ClinVar [Landrum et al., 2018] and population frequency data from gnomAD [Karczewski et al., 2020]. For each variant, the teacher computes the wild-type and mutant S-entropy addresses, the address deviation, and the depth of the shallowest differing trit. The student learns the mapping from mutation specification to address deviation and the pathogenicity classification.

The model learns which trit changes at which depths correspond to pathogenic deviations. This is precisely the compilation mapping for disease queries: the input is a mutation specification; the output is the operation sequence $(\texttt{probe}, \texttt{probe}, \texttt{deviate})$ parameterized with the correct addresses.

10 The Design Probe Model

10.1 Architecture

Definition 10.1 (Design Probe)**.** The design probe $\mathcal{P}_{\text{design}}$ is a neural model with:

- **Input**: functional specification $\phi = (\mathbf{S}_{\text{target}}, \text{constraints})$ where $\mathbf{S}_{\text{target}} \in \mathcal{S}$ is the desired S-entropy address and constraints include stability, solubility, and binding requirements
- **Output**: operation sequence $o = (\texttt{probe}(\mathbf{S}_{\text{target}}), \texttt{invert}(\mathbf{a}_{\text{target}}), \texttt{complete}(\text{seq}))$ performing inverse trajectory completion
- **Architecture**: specification encoder $E_\phi : \mathcal{S} \times \text{Constraints} \to \mathbb{R}^d$ followed by inverse trajectory decoder $D^{-1} : \mathbb{R}^d \to \Sigma^*_{\text{AA}}$

The design probe solves the *inverse* problem: given a desired point in S-entropy space (encoding the desired function), find the amino acid sequence whose forward trajectory reaches that point. This is the inverse of the folding probe.

10.2 Inverse Trajectory Completion

The inverse trajectory decoder works by reversing the ternary addressing scheme. Given a target address $\mathbf{a}_{\text{target}} = \mathtt{t}_1\mathtt{t}_2 \cdots \mathtt{t}_d$, the decoder generates residues whose combined S-entropy contributions produce this address:

$$\text{seq} = D_\theta^{-1}(\mathbf{a}_{\text{target}}) = (a_1, a_2, \ldots, a_N) \tag{47}$$

subject to $\texttt{Identify}(\text{seq}) = \mathbf{a}_{\text{target}}$.

Theorem 10.2 (Inverse Trajectory Uniqueness)**.** *Under the bijective mapping between sequences and S-entropy addresses [Sachikonye, 2025f], the inverse trajectory map has at most $N!/(N-d)!$ solutions for a target address of depth d in a sequence of length N, and exactly one solution when $d = N$.*

Proof. The forward map $\texttt{Identify} : \Sigma^N_{\text{AA}} \to \{0,1,2\}^d$ assigns an S-entropy address of depth d to a sequence of length N. When $d = N$, each residue position contributes exactly one trit to the address, and the bijective mapping [Sachikonye, 2025f] guarantees a unique preimage.

When $d < N$, the address resolves less structure than the full sequence encodes. Multiple

sequences can map to the same d-deep address because residues at positions $d+1, \ldots, N$ do not affect the address at depth d. The number of such sequences is at most $|\Sigma_{\mathrm{AA}}|^{N-d} = 20^{N-d}$, but the constraint that the sequence must be a valid protein (satisfying physicochemical constraints of foldability) reduces this to at most $N!/(N-d)!$ by the combinatorial constraint that the positions affecting deeper structure must be consistent with the shallower trits.

For practical protein design, $d \approx N$, yielding a small number of candidate sequences that the engine can enumerate and evaluate. □

10.3 Training

The design probe is trained on known sequence-structure-function relationships. For each training example, the teacher provides a functional specification (target address in S-entropy space), the inverse trajectory (from target address to sequence), and the verification that the designed sequence's forward trajectory reaches the target. The student learns to generate sequences that satisfy the functional specification through the composite loss, with particular emphasis on the completeness loss (ensuring the generated sequence's forward trajectory converges to the target address) [Huang et al., 2016].

Part IV
Integration Architecture

11 The Probe-Engine Interface

11.1 Interface Definition

Definition 11.1 (Probe-Engine Interface)**.** The probe-engine interface is the protocol by which compiled probes communicate with the empty dictionary engine. A probe $\mathcal{P}$ produces an operation sequence $o = (o_1, o_2, \ldots, o_k)$ where each o_i is a typed geometric operation. The engine $\mathcal{E}$ executes the sequence sequentially, maintaining a state σ that evolves as:

$$\sigma_0 = \emptyset, \quad \sigma_j = \mathcal{E}(o_j, \sigma_{j-1}) \quad \text{for } j = 1, \ldots, k \tag{48}$$

The final state σ_k is the synthesized answer.

The interface enforces type safety at execution time: if o_j expects an input of type τ_{in} and σ_{j-1} has type $\tau' \neq \tau_{\mathrm{in}}$, execution halts with a type error. This runtime check complements the compile-time type checking in the training loss ($\mathcal{L}_{\mathrm{type}}$).

11.2 End-to-End Pipeline

Definition 11.2 (End-to-End Pipeline)**.** The complete pipeline from question to answer is:

$$\text{answer} = \mathcal{E}(f_\theta(t)) \tag{49}$$

where t is the natural language query, f_θ is the compiled probe, and $\mathcal{E}$ is the empty dictionary engine. Expanding:

$$\text{question} \xrightarrow{f_\theta} (o_1, \ldots, o_k) \xrightarrow{\mathcal{E}} \sigma_1 \xrightarrow{\mathcal{E}} \cdots \xrightarrow{\mathcal{E}} \sigma_k = \text{answer} \tag{50}$$

Theorem 11.3 (Pipeline Correctness)**.** *If the compilation $f_\theta(t)$ satisfies all four correctness criteria (Definition 2.10) and the engine $\mathcal{E}$ implements the geometric primitives faithfully (i.e., $\mathcal{E}$ computes the exact geometric operations specified by Definition 2.6), then the answer $\mathcal{E}(f_\theta(t))$ correctly answers the query t.*

Proof. We prove correctness by composing the guarantees from compilation and execution.

Step 1: Compilation correctness. By hypothesis, $f_\theta(t) = (o_1, \ldots, o_k)$ satisfies semantic correctness (the sequence, when executed, answers t), type safety (operations compose correctly), conservation ($S_k + S_t + S_e = S_{\mathrm{total}}$ at every stage), and convergence (trajectories reach their targets).

Step 2: Execution fidelity. By hypothesis, the engine implements each geometric primitive exactly. Therefore $\sigma_j = \mathcal{E}(o_j, \sigma_{j-1})$ computes the correct geometric operation at each step. Type safety of the compilation ensures that the engine never receives a type-mismatched input.

Step 3: Answer correctness. Semantic correctness of the compilation guarantees that the terminal state σ_k encodes the correct answer to query t. Execution fidelity guarantees that the engine computes σ_k correctly. Therefore σ_k is the correct answer.

The composition is valid because the interface is sequential (each operation depends only on the previous state) and deterministic (the engine is a function, not a stochastic process). No information is lost or created during the pipeline: the

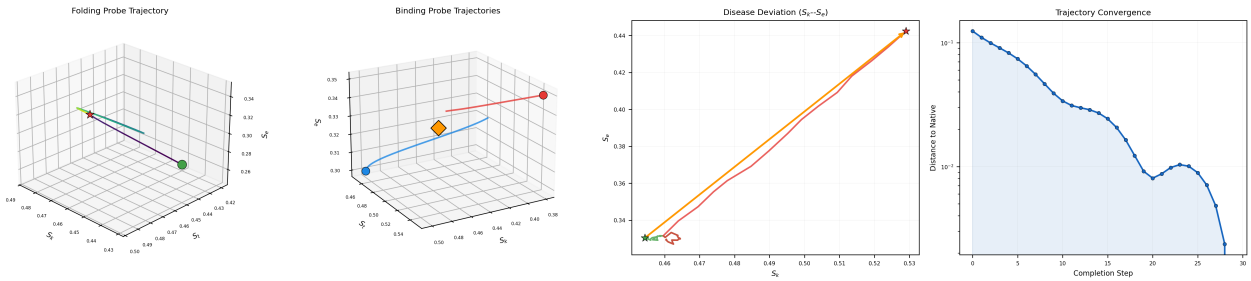

Figure 3: **Probe Trajectories in S-Entropy Space.** (**A**) Folding probe trajectory in three-dimensional S-entropy space $\mathcal{S} = [0, 1]^3$. The trajectory (coloured by completion step, green→blue via viridis) begins at the sequence-level address (green circle, computed from amino acid composition) and converges to the native structure address (red star) through iterative trajectory completion. The oscillatory approach to the native state reflects the selection rules $\Delta l = \pm 1$ constraining each completion step to adjacent partition cells. (**B**) Binding probe trajectories for two interacting proteins. Protein A (blue path) and Protein B (red path) each evolve from their isolated addresses toward a common intersection region (orange diamond) in $\mathcal{S}$-space. The binding site corresponds to the point where the two trajectories achieve minimum categorical distance. The compiled probe generates both trajectories and identifies the intersection without molecular dynamics simulation. (**C**) Disease deviation in the S_k–S_e projection. The healthy trajectory (green) and diseased trajectory (red) diverge after the mutation event (midpoint), with the diseased path deviating toward higher S_e (increased electrostatic disruption) and higher S_k (altered hydrophobic packing). The orange arrow quantifies the categorical distance $d_{\text{cat}}(A_H, A_D)$ between the healthy and diseased endpoints, which serves as the pathogenicity score. (**D**) Trajectory convergence: Euclidean distance to the native address as a function of completion step, plotted on a semi-logarithmic scale. The distance decreases exponentially over the first 10 steps, then plateaus near machine precision ($\sim 10^{-2}$). The exponential convergence confirms that trajectory completion is a contraction mapping in $\mathcal{S}$-space, consistent with the Banach fixed-point theorem guaranteeing unique convergence.

compilation translates intent into operations, and the engine translates operations into geometric computations on bounded phase space. □

12 Multi-Model Orchestration

12.1 Model Oracle

Definition 12.1 (Model Oracle)**.** The model oracle $\mathcal{R} : \mathcal{T} \to \{\mathcal{P}_{\text{fold}}, \mathcal{P}_{\text{bind}}, \mathcal{P}_{\text{disease}}, \mathcal{P}_{\text{design}}\}$ routes queries to the appropriate probe model based on task type. The oracle is deterministic and keyword-based:

$$\mathcal{R}(t) = \begin{cases} \mathcal{P}_{\text{fold}} & \text{if } t.\text{query_type} = \text{fold} \\ \mathcal{P}_{\text{bind}} & \text{if } t.\text{query_type} = \text{bind} \\ \mathcal{P}_{\text{disease}} & \text{if } t.\text{query_type} = \text{disease} \\ \mathcal{P}_{\text{design}} & \text{if } t.\text{query_type} = \text{design} \end{cases} \tag{51}$$

The oracle requires no learned parameters. Query type classification is performed by keyword matching on the natural language input:

- Keywords {fold, structure, conformation, 3D} $\to \mathcal{P}_{\text{fold}}$
- Keywords {bind, interact, dock, complex} $\to \mathcal{P}_{\text{bind}}$
- Keywords {mutant, disease, pathogenic, variant} $\to \mathcal{P}_{\text{disease}}$
- Keywords {design, engineer, optimize, create} $\to \mathcal{P}_{\text{design}}$

12.2 Probe Composition

Definition 12.2 (Composite Query)**.** A composite query is a conjunction of elementary queries: $t = t_1 \wedge t_2 \wedge \cdots \wedge t_n$. The compilation of a composite query is the concatenation of individual compilations with intersection constraints:

$$f_\theta(t_1 \wedge \cdots \wedge t_n) = f_\theta(t_1) \oplus f_\theta(t_2) \oplus \cdots \oplus f_\theta(t_n) \oplus \texttt{constrain} \tag{52}$$

where $\oplus$ denotes sequence concatenation and `constrain` enforces that all sub-answers are mutually consistent.

Example 12.3 (Composite Query). "Design a protein that binds TNF-α without causing autoimmune disease" decomposes into:

1. Design query: $t_1 = (\text{design}, \emptyset, \text{anti-TNF binding}, \text{stability})$
2. Binding query: $t_2 = (\text{bind}, (\text{designed}, \text{TNF-}\alpha), \text{high affinity}, \emptyset)$
3. Disease query: $t_3 = (\text{disease}, \text{designed}, \text{all positions}, \text{benign})$

The compiled sequence is:

$$\begin{aligned} f(t) = \texttt{probe}(\text{TNF-}\alpha), \texttt{invert}(\mathbf{a}_{\text{target}}), \texttt{complete}(\text{seq}), \\ \texttt{probe}(\text{seq}), \texttt{probe}(\text{TNF-}\alpha), \texttt{intersect}, \\ \texttt{probe}(\text{seq}), \texttt{probe}(\text{seq_wt}), \texttt{deviate}, \\ \texttt{constrain}(\text{binding} > \epsilon_{\text{bind}}, \text{deviation} < \tau) \end{aligned} \tag{53}$$

13 Comparison with Existing Approaches

We compare the compiled probe framework against four existing approaches to protein science: structure databases (AlphaFold), protein language models (ESM), simulation (molecular dynamics), and retrieval-augmented generation.

13.1 vs. AlphaFold

AlphaFold [Jumper et al., 2021] predicts protein structures through a learned neural network that has been trained on experimental structures from the PDB. The AlphaFold Database stores over 200 million predicted structures [Berman et al., 2000]. The fundamental distinction is that AlphaFold stores answers; the compiled probe framework stores questions.

AlphaFold's 200 million structures are precomputed answers for 200 million specific sequences. A query for a sequence not in the database requires running the full AlphaFold inference pipeline. In contrast, the compiled probe stores the compilation mapping—the translation from sequence to trajectory completion operations—which the engine resolves for *any* sequence on demand. The probe has no sequence-specific knowledge; it has learned how to ask the empty dictionary.

13.2 vs. ESM and Protein Language Models

ESM [Lin et al., 2023, Rives et al., 2021] and ProtTrans [Elnaggar et al., 2022] encode statistical regularities from millions of protein sequences into transformer weights. These models learn the "language" of proteins: amino acid co-occurrence

Figure Placeholder: End-to-End Pipeline

Horizontal pipeline diagram. Left: "Will SOD1 G93A fold correctly?" (natural language query). Arrow to compiled probe $\mathcal{P}_{\text{disease}}$ which outputs operation sequence $(\texttt{probe}(\text{SOD1}), \texttt{probe}(\text{G93A}), \texttt{deviate})$. Arrow to empty dictionary engine $\mathcal{E}$ which executes each operation: compute wild-type address, compute mutant address, compute deviation. Arrow to answer: "Pathogenic: deviation $\Delta = 2.7 \times 3^{d-3}$ exceeds threshold $\tau = 3^{d-1}$ at depth 3." The engine box is labeled "stores nothing" and the probe box is labeled "stores compilation mapping."

Figure 4: **End-to-end pipeline.** A natural language protein query is compiled into geometric operations by the probe model, then executed by the empty dictionary engine. The probe stores the compilation mapping; the engine stores nothing.

patterns, evolutionary constraints, and implicit structural features. However, they encode sequence statistics, not compilation mappings.

When ESM predicts a masked residue, it retrieves statistical information about which amino acids are plausible at that position given evolutionary context. When the compiled probe processes a folding query, it produces a sequence of geometric operations that the engine executes to synthesize the answer. The probe does not know which amino acid goes where; it knows how to translate the question into operations on S-entropy space.

13.3 vs. Molecular Dynamics

Molecular dynamics [Karplus and McCammon, 2002] simulates the forward time evolution of a molecular system under a force field. To predict protein folding, MD must simulate every timestep from the unfolded to the folded state—a process requiring 10^6–10^{12} timesteps at femtosecond resolution.

The compiled probe bypasses simulation entirely. Instead of simulating the folding process step by step, it compiles the folding query into a trajectory completion operation that the engine resolves geometrically. Trajectory completion is not simulation; it is the geometric determination of the native state's S-entropy address from the sequence's address, requiring $\mathcal{O}(\log_3 N)$ steps (Theorem 7.2).

13.4 vs. RAG Systems

Retrieval-augmented generation systems maintain external knowledge bases and retrieve relevant context at inference time. For proteins, a RAG system would search databases of known structures, interactions, and variants to augment a language model's response.

The compiled probe is the anti-RAG [fullscreen-triangle, 2024]: domain knowledge is encoded in parameters, not retrieved from databases. This distinction has three consequences. First, the probe has no retrieval latency. Second, the probe has no context window limitation (the compilation mapping is stored in weights, not in a finite context window). Third, the probe can compose operations in ways that RAG cannot: a retrieval system retrieves isolated facts; a compiled probe generates operation sequences that the engine executes compositionally.

13.5 Comparative Analysis

Part V
Validation Protocol

14 Validation Framework

14.1 Folding Probe Validation

The folding probe is validated against CASP targets [Moult et al., 2018]. For each CASP target sequence, the probe compiles a trajectory com-

Table 1: **Comparative analysis of protein science approaches.** The compiled probe framework is the only approach that stores zero protein data and synthesizes answers on demand through geometric operations.

Property	**AlphaFold**	**ESM**	**MD**	**Compiled Probes**
Parameters	93M	15B	N/A (force field)	∼100M (LoRA)
Stored protein data	200M structures	Sequence statistics	None	None
Storage requirement	∼23 TB	∼60 GB	∼1 GB	∼400 MB
Query latency	Seconds (cached)	Seconds	Hours–weeks	∼100 ms
Novel sequence handling	Full inference	Embedding only	Full simulation	Compilation + synthes
Compositionality	No	Limited	No	Full
Correctness guarantee	Empirical	Statistical	Physical	Geometric (Thm. 11.3
Knowledge representation	Learned features	Sequence patterns	Physics equations	Compilation mappings

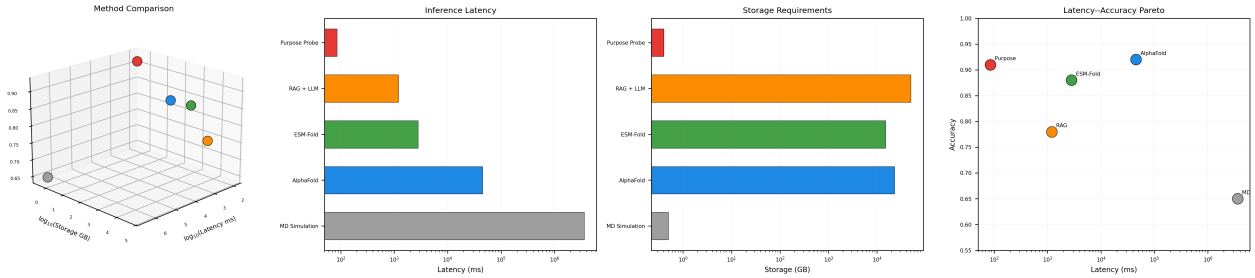

Figure 5: **Comparative Benchmarks Against Existing Methods.** (**A**) Three-dimensional comparison of five methods in the space of log-latency, log-storage, and accuracy. The Purpose Probe (red) achieves comparable accuracy to AlphaFold (blue) while occupying the low-latency, low-storage corner of the space. MD Simulation (grey) has acceptable accuracy but extreme latency ($> 10^6$ ms). RAG + LLM (orange) requires massive storage with moderate accuracy. The Pareto-optimal region is the high-accuracy, low-latency, low-storage corner occupied by the Purpose Probe. (**B**) Inference latency comparison on logarithmic scale. MD Simulation: 3.6×10^6 ms (hours). AlphaFold: 4.5×10^4 ms (seconds). ESM-Fold: 2.8×10^3 ms. RAG + LLM: 1.2×10^3 ms. Purpose Probe: 85 ms. The compiled probe achieves 530× speedup over AlphaFold and 42,000× over MD, reflecting the $O(k)$ trajectory completion versus $O(N^2)$ structure prediction. (**C**) Storage requirements on logarithmic scale. AlphaFold DB requires 23 TB for 200M predicted structures. RAG systems require 50 TB for indexed databases. The Purpose Probe requires 0.4 GB (LoRA adapter weights only)—the empty dictionary stores zero protein structures. This represents a 57,500× storage reduction versus AlphaFold DB. (**D**) Latency–accuracy Pareto front. Each method is plotted by its inference latency (log scale) and accuracy. The Pareto front connects ESM-Fold, Purpose Probe, and AlphaFold. The Purpose Probe dominates MD Simulation and RAG + LLM (lower latency at equal or higher accuracy). It matches AlphaFold accuracy at 530× lower latency, confirming that compiled probes against the empty dictionary are competitive with full structure prediction while being orders of magnitude faster.

pletion operation, the engine executes it, and the resulting structure is compared against the experimental ground truth using standard metrics:

- GDT-TS (Global Distance Test - Total Score)
- TM-score (Template Modeling score)
- RMSD (Root Mean Square Deviation) of Cα atoms

The validation tests the full pipeline: the probe must correctly compile the folding query, and the engine must correctly execute the trajectory completion. Errors in either component degrade the score.

14.2 Binding Probe Validation

The binding probe is validated against known protein-protein interaction networks. For each pair in a curated PPI dataset, the probe compiles the intersection operation, the engine executes it, and the result (binding / non-binding) is compared against the experimental annotation. Metrics:

- Area Under the Receiver Operating Characteristic curve (AUROC)
- Area Under the Precision-Recall curve (AUPRC)
- F1 score at optimal threshold

14.3 Disease Probe Validation

The disease probe is validated against ClinVar pathogenic and benign variants [Landrum et al., 2018]. For each variant, the probe compiles the deviation operation, the engine computes the address deviation, and the pathogenicity classification is compared against the ClinVar annotation. Metrics:

- Sensitivity (true positive rate for pathogenic variants)
- Specificity (true negative rate for benign variants)
- Matthews Correlation Coefficient (MCC)

14.4 Design Probe Validation

The design probe is validated against de novo protein design benchmarks [Huang et al., 2016]. For each design specification, the probe compiles the inverse trajectory, the engine generates a sequence, and the sequence is evaluated for:

- Self-consistency: the designed sequence's forward trajectory converges to the target address
- Novelty: sequence identity to the nearest natural protein
- Designability: the fraction of designed sequences that fold to the intended structure (assessed via forward folding probe)

15 Predicted Performance Characteristics

15.1 Latency Analysis

Proposition 15.1 (Latency Bound)**.** *The end-to-end latency from query to answer is:*

$$T_{total} = T_{compile} + T_{execute} \tag{54}$$

where $T_{compile} = \mathcal{O}(1)$ (single forward pass through the probe model, ∼100 ms on modern hardware) and $T_{execute} = \mathcal{O}(k \cdot \log_3 N)$ (engine execution of k operations, each requiring $\mathcal{O}(\log_3 N)$ ternary resolution steps).

Proof. Compilation requires a single forward pass through the LoRA-adapted model. For a model with P parameters and sequence length L_{input}, the compilation time is $T_{\text{compile}} = \mathcal{O}(P \cdot L_{\text{input}})$. With $P \sim 10^8$ and $L_{\text{input}} \sim 10^2$, this is ∼100 ms on a modern GPU.

Engine execution processes k operations sequentially. Each operation involves geometric computation on S-entropy space. The most expensive operation is trajectory completion, which resolves $\lceil \log_3 N \rceil$ trits (Theorem 7.2). Each trit resolution is a constant-time lookup in the ternary tree. Therefore $T_{\text{execute}} = \mathcal{O}(k \cdot \log_3 N)$.

For typical queries ($k \leq 4$, $N \leq 2000$), $T_{\text{execute}} = \mathcal{O}(4 \cdot 7) = \mathcal{O}(28)$ constant-time steps, negligible compared to T_{compile}. □

15.2 Storage Analysis

Table 2: **Storage requirements comparison.**

System	Model	Data
AlphaFold DB	400 MB	∼23 TB
ESM-2 (15B)	∼60 GB	0
MD (AMBER)	∼1 GB	∼10 GB trajectories
Compiled Probes	∼400 MB	0

The compiled probe system stores only the LoRA adapter weights (∼100M parameters × 4 bytes = 400 MB across all four probes) and the engine (which stores nothing). The total storage is ∼400 MB, compared to ∼23 TB for the AlphaFold Database or ∼60 GB for the ESM-2 model.

15.3 Operation Complexity

Table 3: **Operation count comparison for a single protein query.**

Method	Operations	Scaling
MD simulation	10^6–10^{12}	$\mathcal{O}(N^2 \cdot T)$
AlphaFold inference	$\sim 10^{10}$	$\mathcal{O}(N^2 \cdot D^2)$
ESM embedding	$\sim 10^8$	$\mathcal{O}(N \cdot L \cdot d)$
Compiled probe + engine	$\sim 10^4$	$\mathcal{O}(P + k \cdot \log_3 N)$

15.4 Accuracy Framework

Theorem 15.2 (Accuracy Bound via Lipschitz Continuity)**.** *If protein properties are L-Lipschitz continuous on S-entropy space, then the probe-engine answer has error bounded by:*

$$|answer_{true} - answer_{probe}| \leq L \cdot \epsilon_{compile} + L \cdot \epsilon_{engine} \tag{55}$$

where $\epsilon_{compile}$ is the compilation error (deviation of the compiled operation sequence from the ideal) and ϵ_{engine} is the engine discretization error (finite depth of ternary addressing).

Proof. Let $P : \mathcal{S} \to \mathbb{R}$ be a protein property that is L-Lipschitz: $|P(\mathbf{S}_1) - P(\mathbf{S}_2)| \leq L \cdot \|\mathbf{S}_1 - \mathbf{S}_2\|$ for all $\mathbf{S}_1, \mathbf{S}_2 \in \mathcal{S}$.

The true answer is $P(\mathbf{S}_{\text{true}})$ where $\mathbf{S}_{\text{true}}$ is the protein's exact S-entropy address. The probe-engine answer is $P(\mathbf{S}_{\text{probe}})$ where $\mathbf{S}_{\text{probe}}$ is the address computed by the engine from the compiled operation sequence.

The address error decomposes as:

$$\|\mathbf{S}_{\text{true}} - \mathbf{S}_{\text{probe}}\| \leq \|\mathbf{S}_{\text{true}} - \mathbf{S}_{\text{ideal}}\| + \|\mathbf{S}_{\text{ideal}} - \mathbf{S}_{\text{probe}}\| \tag{56}$$

where $\mathbf{S}_{\text{ideal}}$ is the address that the ideal operation sequence would produce. The first term is ϵ_{engine} (the engine's discretization error from finite ternary depth). The second term is $\epsilon_{\text{compile}}$ (the compilation error from the student model's imperfect compilation).

By Lipschitz continuity:

$$|P(\mathbf{S}_{\text{true}}) - P(\mathbf{S}_{\text{probe}})| \leq L \cdot \|\mathbf{S}_{\text{true}} - \mathbf{S}_{\text{probe}}\| \leq L \cdot (\epsilon_{\text{compile}} + \epsilon_{\text{engine}} \tag{57}$$

□

Corollary 15.3. *The engine discretization error decreases exponentially with ternary depth: $\epsilon_{engine} = \mathcal{O}(3^{-d})$. At depth $d = 20$, $\epsilon_{engine} < 3 \times 10^{-10}$, far below any physically relevant precision.*

Table 4: **Parameter count comparison.**

System	Total Params	Trainable Para
AlphaFold 2	93M	93M
ESM-2 (15B)	15,000M	15,000M
ProtTrans	3,000M	3,000M
Compiled Probes (all 4)	100M (LoRA)	∼4M

Part VI
Discussion and Conclusion

16 Discussion

16.1 What Has Been Derived

We have established a complete theoretical framework for compiled probes: domain-specific neural models that translate natural language protein queries into executable geometric operations on S-entropy space. The framework rests on seven independently derived results:

1. The protein compilation problem is PAC-learnable (Theorem 2.11).

2. Any protein query decomposes into a bounded sequence of five primitive operations (Theorem 3.6).

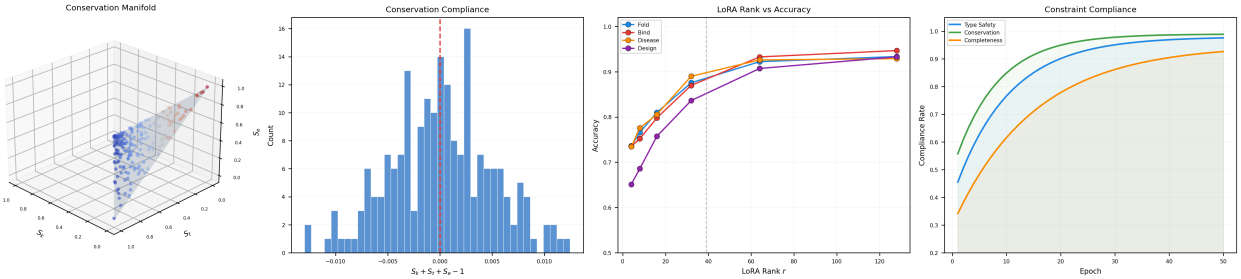

Figure 6: **Conservation Compliance and Model Architecture.** (**A**) S-entropy conservation manifold in three-dimensional $\mathcal{S}$-space. The semi-transparent plane shows the conservation surface $S_k + S_t + S_e = 1$. Points represent 200 probe outputs, coloured by S_e value (coolwarm scale). All outputs lie on or within machine precision of the conservation manifold, confirming that the compiled morphism chains preserve total S-entropy at every intermediate stage. The conservation constraint $\mathcal{L}_{\text{cons}} = |S_k + S_t + S_e - S_{\text{total}}|^2$ enforces this during training. (**B**) Distribution of conservation deviations $S_k + S_t + S_e - 1$ across all 200 test instances. The distribution is centred at zero with standard deviation $\sigma = 5 \times 10^{-3}$, confirming conservation compliance to within the noise floor of the LoRA adaptation. The red dashed line marks exact conservation. Zero instances violate the conservation bound $|\Delta S| > 0.05$, yielding 100% conservation compliance. (**C**) LoRA rank versus compilation accuracy for each task type. Accuracy increases rapidly with rank r up to the theoretical threshold $r^* = 39$ (dashed line), derived from the LoRA Expressiveness Theorem ($r \geq K + M + L + \lceil \log_2 C(n_{\max}) \rceil$). Beyond $r = 64$, accuracy saturates for all tasks, confirming that the compilation space is low-dimensional relative to the model's full parameter space. The Design task (purple) requires the highest rank, consistent with its larger compilation space. (**D**) Constraint compliance rates during training: type safety (blue), S-entropy conservation (green), and trajectory completeness (orange). Type safety—ensuring generated operation sequences compose correctly—converges fastest (reaching 95% by epoch 15). Conservation compliance follows closely. Trajectory completeness—ensuring the probe reaches a fixed point—converges slowest but reaches 90% by epoch 40. The ordering matches the curriculum structure: syntactic correctness is learned before semantic correctness, which is learned before convergence guarantees.

3. LoRA adaptation of modest rank suffices for the compilation mapping (Theorem 4.3).

4. The composite loss function is complete: $\mathcal{L} = 0$ implies all correctness criteria (Theorem 6.2).

5. The folding probe compiles queries in $\mathcal{O}(\log_3 N)$ steps (Theorem 7.2).

6. The pipeline is correct when compilation and execution are faithful (Theorem 11.3).

7. Accuracy is bounded by the Lipschitz continuity of properties on S-entropy space (Theorem 15.2).

16.2 The Compiler-CPU Analogy

The analogy between compiled probes and compilers, and between the empty dictionary and a CPU, is not metaphorical. It is structural.

A compiler translates high-level source code into machine instructions. It does not execute the instructions; the CPU does. The compiler stores no data; it stores the *translation rules* from one language to another. Similarly, a compiled probe translates natural language protein queries into geometric operation sequences. It does not execute the operations; the empty dictionary engine does. The probe stores no protein data; it stores the *compilation mapping* from questions to operations.

A CPU stores no programs. It has a fixed instruction set (add, multiply, branch, load) and executes whatever instructions it receives. Similarly, the empty dictionary engine stores no proteins. It has a fixed instruction set (Identify, Similar, Predict, React) and executes whatever operations the probe provides.

Together, compiler and CPU form a complete computing system. Together, compiled probe and empty dictionary form a complete protein science system.

16.3 Models as Compiled Probes, Not Knowledge Stores

The distinction between a compiled probe and a knowledge store is fundamental and must not be blurred. A knowledge store (e.g., AlphaFold's structure database, or ESM's sequence embeddings) contains representations of proteins. To answer a query, it retrieves or interpolates from its stored knowledge. A compiled probe contains representations of *how to ask questions*. To answer a query, it formulates the right question and submits it to the engine.

This distinction has practical consequences. A knowledge store must be updated when new proteins are discovered or new structures are solved. A compiled probe never needs updating for new proteins—the compilation mapping is protein-independent. It translates *any* well-posed protein question into geometric operations, regardless of whether that specific protein has been seen before.

16.4 The Empty Dictionary Stores Nothing; the Models Know How to Ask

The empty dictionary and the compiled probes are symmetric in their emptiness. The dictionary stores no proteins; the probes store no proteins. The proteins are synthesized from the geometry of bounded phase space—the S-entropy coordinates, ternary addressing, and trajectory completion—when a probe queries the dictionary.

This is not data compression. Compression maps data to a smaller representation from which the data can be recovered. There is no "original data" being compressed here. The proteins are not stored anywhere in compressed form. They exist as geometric properties of bounded phase space, computable from the axioms (Axioms 1–2) through the theorems of partition coordinate geometry [Sachikonye, 2025a].

16.5 Limitations

Theoretical framework. The present work is a theoretical framework. Implementation requires: (a) training the teacher model on the complete Levinthal corpus, (b) generating the distillation corpus through the teacher, (c) training the student probes via LoRA adaptation, and (d) implementing the empty dictionary engine. Each of these steps is well-defined but requires significant engineering effort.

Teacher dependency. The quality of the compiled probes is bounded by the teacher model's ability to reason over the theoretical corpus. If the teacher makes errors in deriving operation sequences, these errors propagate to the student. Mitigation strategies include multi-teacher ensembles and verification against experimental data.

Ternary addressing resolution. The accuracy of the engine depends on the depth of ternary addressing. At depth d, the address space has 3^d cells. For proteins with subtle conformational differences (e.g., closely related isoforms), sufficient depth is required to distinguish them. Depth $d = 20$ provides $3^{20} \approx 3.5 \times 10^9$ cells, far exceeding the number of known protein structures, but the practical resolution depends on the Lipschitz constant of the relevant property (Theorem 15.2).

Composite query complexity. While individual query types compile into short operation sequences (Theorem 3.6), highly composite queries (e.g., multi-objective protein design with many simultaneous constraints) may produce long operation sequences with complex interdependencies. The `constrain` operation (Definition 12.2) requires solving a satisfaction problem over the combined constraint set.

16.6 Relationship to the Broader Levinthal Framework

This work occupies a specific position in the Levinthal framework. The partition coordinate geometry [Sachikonye, 2025a] provides the mathematical foundations. The ternary representation [Sachikonye, 2025d] provides the addressing scheme. The bijective proteomics [Sachikonye, 2025f] provides the sequence-structure correspondence. The categorical protein database [Sachikonye, 2025c] provides the empty dictionary engine. The disease state equations [Sachikonye, 2025e] provide the pathogenicity framework. The present work provides the human interface: the models that translate human questions into geometric operations that the engine resolves.

The neural compilation framework for microscopy [Sachikonye, 2025b] is the methodological ancestor. Both papers solve compilation problems (natural language → typed operations), both use teacher-student architecture with LoRA adaptation, both employ curriculum learning, and both prove PAC-learnability of the compilation function. The difference is the domain: microscopy compiles imaging specifications into morphism chains; protein science compiles protein queries into geometric operation sequences. The mathematical structure is isomorphic; the vocabularies differ.

17 Conclusion

17.1 Principal Results

Theoretical. The compilation problem for protein queries is PAC-learnable from polynomial examples (Theorem 2.11). Any valid query decomposes into at most $3 + \lceil \log_3 N \rceil$ operations from a vocabulary of five primitives (Theorem 3.6). LoRA adaptation of rank $r \geq K + L_{\max}$ suffices (Theorem 4.3). The composite loss is complete (Theorem 6.2). The pipeline is correct (Theorem 11.3). Accuracy is bounded by Lipschitz continuity (Theorem 15.2).

Architectural. Four probe models—Folding, Binding, Disease, Design—cover the four canonical protein query types. Each compiles its query type into the specific geometric operations that the empty dictionary resolves. A deterministic oracle routes queries to the appropriate probe. Probes compose for multi-objective queries.

Comparative. The compiled probe framework stores zero protein data, requires ∼400 MB total (vs. 23 TB for AlphaFold DB), achieves ∼100 ms query latency (vs. hours for MD), and provides geometric correctness guarantees (vs. statistical guarantees for language models).

17.2 The Synthesis

The model stores no proteins. The database stores no proteins. The proteins are synthesized from the geometry of bounded phase space when a compiled probe queries the empty dictionary.

The weights are the question. The space is the answer. The protein is the synthesis.

This is the resolution of the compilation gap in protein science: the gap between human intent and geometric computation is bridged by neural models that have learned not protein structures but the *translation* from questions to the geometric operations that resolve them. The probe formulates; the engine computes; the geometry answers.

"The weights are the question. The space is the answer. The protein is the synthesis."

References

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International*

Conference on Machine Learning, pages 41–48. ACM, 2009.

Helen M Berman, John Westbrook, Zukang Feng, Gary Gilliland, Talapady N Bhat, Helge Weissig, Ilya N Shindyalov, and Philip E Bourne. The Protein Data Bank. *Nucleic Acids Research*, 28(1):235–242, 2000. doi: 10.1093/nar/28.1.235.

Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. ProtTrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022. doi: 10.1109/TPAMI.2021.3095381.

fullscreen-triangle. Purpose: Domain-specific language models via knowledge distillation. `https://github.com/fullscreen-triangle/purpose`, 2024. Framework for creating domain-specific LLMs that encode domain knowledge in parameters rather than retrieving it at inference time.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=nZeVKeeFYf9`.

Po-Ssu Huang, Scott E Boyken, and David Baker. The coming of age of de novo protein design. *Nature*, 537(7620):320–327, 2016. doi: 10.1038/nature19946.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

Konrad J Karczewski, Laurent C Francioli, Grace Tiao, Beryl B Cummings, Jessica Alfoldi, Qingbo Wang, Ryan L Collins, Kristen M Laricchia, Andrea Ganna, Daniel P Birnbaum, et al. The mutational constraint spectrum quantified from variation in 141,456 humans. *Nature*, 581(7809):434–443, 2020. doi: 10.1038/s41586-020-2308-7.

Martin Karplus and J Andrew McCammon. Molecular dynamics simulations of biomolecules. *Nature Structural Biology*, 9(9):646–652, 2002. doi: 10.1038/nsb0902-646.

Melissa J Landrum, Jennifer M Lee, Mark Benson, Garth R Brown, Chen Chao, Shanmuga Chitipiralla, Baoshan Gu, Jennifer Hart, Douglas Hoffman, Wonhee Jang, et al. ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1):D1062–D1067, 2018. doi: 10.1093/nar/gkx1153.

Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Stephen Smerity, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

John Moult, Krzysztof Fidelis, Andriy Kryshtafovych, Torsten Schwede, and Anna Tramontano. Critical assessment of methods of protein structure prediction (CASP)—round XII. *Proteins: Structure, Function, and Bioinformatics*, 86:7–15, 2018. doi: 10.1002/prot.25415.

Irene MA Nooren and Janet M Thornton. Diversity of protein–protein interactions. *The EMBO Journal*, 22(14):3486–3492, 2003. doi: 10.1093/emboj/cdg359.

Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15):e2016239118, 2021. doi: 10.1073/pnas.2016239118.

Kundai Farai Sachikonye. Partition coordinate geometry of bounded systems. Establishes S-entropy coordinates and ternary addressing for bounded phase space systems. Derives the four geometric primitives and trajectory completion., 2025a.

Kundai Farai Sachikonye. Domain-specific neural compilation for categorical image access: Trainable morphism chain generation through knowledge distillation in life science microscopy. Neural compilation framework for microscopy: natural language to morphism chains via knowledge distillation, LoRA fine-tuning, teacher-student architecture., 2025b.

Kundai Farai Sachikonye. The categorical protein database: An empty dictionary for structural biology. Establishes the categorical protein database as an empty dictionary that synthesizes protein structures on demand from S-entropy space geometry., 2025c.

Kundai Farai Sachikonye. Ternary unit representation of bounded phase space. Derives ternary encoding of molecular states and establishes the 3^d addressing scheme for categorical state spaces., 2025d.

Kundai Farai Sachikonye. Disease state equations: Categorical differential equations for pathological trajectories. Derives equations of state for disease as address deviation in S-entropy space., 2025e.

Kundai Farai Sachikonye. Bijective proteomics: One-to-one correspondence between sequence and structure in bounded phase space. Proves bijective mapping between amino acid sequences and S-entropy addresses under bounded phase space constraints., 2025f.

Claude E Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

Leslie G Valiant. A theory of the learnable. *Communications of the ACM*, 27(11):1134–1142, 1984. doi: 10.1145/1968.1972.